

Originalmente, en React, la creación de componentes con capacidades avanzadas como el manejo de estado y el acceso a ciclos de vida solo era posible a través de componentes de clase. Sin embargo, la versión 16.8 de React marcó un cambio significativo con la introducción de los *hooks*. Estas nuevas **funciones** permiten a los componentes funcionales acceder a las mismas capacidades que antes estaban limitadas a los componentes de clase, combinan las capacidades completas de React con una sintaxis más sencilla y una mayor facilidad de uso.

¿Qué son los *hooks*?

Los *hooks* son funciones de JavaScript que permiten a los componentes funcionales acceder a características antes reservadas para los componentes de clase. Esto incluye el manejo de estado, la ejecución de efectos secundarios o el acceso al contexto de React, temas que trataremos más adelante. Los *hooks* no solo equiparan a los componentes funcionales con las mismas funcionalidades de los componentes de clase, sino que también ofrecen una escritura y mantenimiento más sencillos.

Algunas de las ventajas de los *hooks* frente a los componentes de clase:

- Son más concisos y fáciles de leer.
- Son más fáciles de probar.
- Son más fáciles de mantener.

Reglas de los *hooks*

Para utilizar correctamente los *hooks* en React, es importante seguir estas reglas establecidas:

1. **Ámbito de uso:** Los *hooks* solo deben utilizarse dentro de componentes funcionales o en *hooks personalizados*. Profundizaremos en los *hooks personalizados* más adelante.
2. **Llamada en el nivel superior:** Siempre deben llamarse en el nivel superior de un componente. Esto significa que no pueden estar dentro de bucles, condiciones o funciones anidadas.
3. **Convención de nombres:** Los nombre de los *hooks*, por convención, empiezan por '*use*' (como *useState* o *useEffect*), para poder diferenciarlos del resto de funciones regulares en JavaScript.

Hooks integrados en React

React ofrece una cantidad de *hooks* incorporados, cada uno con distintos propósitos y niveles de uso frecuente. En las siguientes secciones, trataremos los *hooks* más importantes.

A modo de referencia rápida, aquí tienes una tabla resumen con la mayoría de los *hooks* disponibles en React:

Categoría	Hook	Descripción
Básicos	<i>useState</i>	Manejo del estado local en componentes.
	<i>useEffect</i>	Ejecución de efectos secundarios en componentes funcionales.

Categoría	Hook	Descripción
	<code>useContext</code>	Manejo del contexto en componentes funcionales.
Adicionales	<code>useReducer</code>	Manejo de lógica de estado más compleja y transiciones de estado predecibles.
	<code>useCallback</code>	Devuelve una función memorizada para evitar renderizados innecesarios.
	<code>useMemo</code>	Memorización de valores computados costosos.
	<code>useRef</code>	Acceso a elementos del DOM o componentes React sin provocar un nuevo renderizado.
	<code>useImperativeHandle</code>	Personalización de la instancia de un componente expuesta a componentes padres.
	<code>useLayoutEffect</code>	Se dispara síncronamente después de que los cambios en el DOM se hayan reflejado.
	<code>useDebugValue</code>	Muestra una etiqueta en el inspector de React DevTools para <i>hooks</i> personalizados.
	<code>useDeferredValue</code>	Diferir el cambio de un valor que podría causar componentes "pesados" a renderizar.
	<code>useTransition</code>	Transición de estado más suave para evitar renderizados "pesados".
	<code>useId</code>	Genera un ID único para componentes, útil para relacionar etiquetas con elementos de entrada.

Los tres primeros son los más comunes y fundamentales para entender React. Los *hooks* adicionales ofrecen utilidades más especializadas y pueden o no ser necesarios en tu proyecto.

Los otros diez *hooks* se denominan "*Additional hooks*" en la documentación de *React*. Estos son *hooks* que puede que uses ocasionalmente (o nunca) o que son variaciones de los tres *hooks* básicos. Sin embargo, hay algunas cosas realmente útiles (y un par que pueden considerarse importantes) en este conjunto de *hooks*.

Tip: Para obtener más información sobre los *hooks* de React, consulta la [documentación oficial de React](#).

 [Volver al índice principal](#)

A menudo, los componentes necesitan volver a renderizarse para reflejar cambios en su contenido. Por ejemplo, supón que quieres mostrar un contador y, cada vez que se haga clic en un botón, dicho contador aumente. Con los conocimientos adquiridos hasta ahora, podrías hacer algo así:

```
function App() {
  // Creamos variable counter
  let counter = 0;

  return (
    <main>
      { /* Mostramos el valor de counter en un párrafo */ }
      <p>Contador: {counter}</p>

      <button
        onClick={() => {
          // Cuando se hace click en el botón, le sumamos uno a counter
          counter++;
          // Imprimimos por consola el nuevo valor de counter
          console.log(counter);
        }}
      >
        Click me!
      </button>
    </main>
  );
}

export default App;
```

Si ejecutas este código, comprobarás en la consola del navegador como efectivamente la variable **counter** va aumentando al hacer *clic*, pero en el párrafo con el contador seguirá renderizándose un 0.

¿Por qué sucede esto? React renderiza los componentes una sola vez, por lo que aunque posteriormente cambien valores como el de **counter**, el componente **App** no se vuelve a renderizar y no se pinta en pantalla el nuevo valor del contador.

¿Cómo puedes solucionarlo? Pues utilizando **estados**. Un **estado** es un valor, que cuando es modificado, provoca que el componente se vuelva a renderizar. Por ejemplo, si **counter** fuese un **estado**, al modificarlo, se volvería a renderizar el componente **App** y verías el valor actualizado de **counter** en el párrafo.

Tip: Inicialmente, la creación de **estados** era una funcionalidad exclusiva de los componentes de clase, pero gracias a los *hooks*, ahora puedes crearlos también en componentes funcionales.

Uso básico del *hook* **useState**

Para crear un nuevo estado, primero debes importar el *hook* **useState** de *React*:

```
import { useState } from "react";
```

Tras importarlo, debes llamar al *hook* en el primer nivel del componente, siguiendo la regla de los *hooks* mencionada en la sección anterior.

Cuando utilices `useState`, asigna el valor inicial del **estado** como parámetro. Si quieres crear un estado para almacenar el valor del contador, puedes darle un valor inicial de 0.

App.jsx

```
import { useState } from "react";

function App() {
  // Guardamos en una variable "x" lo que nos devuelve useState(0)
  const x = useState(0);

  // Imprimimos por consola lo que nos ha devuelto useState
  console.log(x);

  return (
    <main>
      <p>Contador: </p>

      <button>Click me!</button>
    </main>
  );
}

export default App;
```

Al inspeccionar la variable `x` en la consola, verás que `useState` devuelve un array con dos elementos: `[0, f]`. El primer elemento es el valor actual del **estado** (0) y el segundo es una función para actualizar ese estado.

```
[0, f];
```

Es habitual desestructurar el array retornado por `useState`. Puedes almacenar el valor del estado en una variable, como `counter`, y nombrar la función para actualizar este estado con un prefijo 'set', seguido del nombre de la variable de estado. En el caso del ejemplo, la llamamos `setCounter`. Así se ve el código con esta práctica:

App.jsx

```
import { useState } from "react";
```

```
function App() {
  // Hacemos destructuring del array que nos devuelve useState
  const [counter, setCounter] = useState(0);

  return (
    <main>
      {/* Mostramos el valor de counter en un párrafo */}
      <p>Contador: {counter}</p>

      <button>Click me!</button>
    </main>
  );
}

export default App;
```

Al ejecutar la aplicación, el contador se mostrará inicialmente con su valor por defecto que es 0. Ahora, vamos a agregar la funcionalidad para actualizar el **estado** `counter` al hacer clic en el botón, utilizando la función de actualización proporcionada por `useState`.

Para modificar el **estado**, no puedes hacer una reasignación a la variable `counter` como se hace típicamente en JavaScript. Por ejemplo, **no es correcto** hacer algo como esto:

```
counter++;

// Lo anterior es un atajo de
counter = counter + 1;
```

O como:

```
counter = 10;
```

Para modificar un **estado**, **siempre hay que utilizar la función retornada por el `useState`**, en este caso `setCounter`. Simplemente llama a la función y le pasas por parámetro el nuevo valor que quieres que tenga el **estado**.

```
setCounter(counter + 1);
```

Si realizas esta modificación en el `onClick` del botón, quedaría algo así:

App.jsx

```
import { useState } from "react";

function App() {
  // Hacemos destructuring del array que nos devuelve useState
  const [counter, setCounter] = useState(0);

  return (
    <main>
      {/* Mostramos el valor de counter en un párrafo */}
      <p>Contador: {counter}</p>

      <button
        onClick={() => {
          // Cuando se hace click en el botón, modificamos el estado de counter
          setCounter(counter + 1);
        }}
      >
        Click me!
      </button>
    </main>
  );
}

export default App;
```

Ahora, si haces clic en el botón, comprobarás como se muestra correctamente el valor actualizado del contador.

Es muy importante que entiendas que **cada vez que cambia un estado en un componente, se vuelve a *renderizar***, es decir, ejecutar. Puedes comprobarlo si añades un `console.log` en la lógica del componente:

App.jsx

```
import { useState } from "react";

function App() {
  // Hacemos destructuring del array que nos devuelve useState
  const [counter, setCounter] = useState(0);

  console.log("render");

  return (
    <main>
      {/* Mostramos el valor de counter en un párrafo */}
      <p>Contador: {counter}</p>

      <button
        onClick={() => {
```

```

        // Cuando se hace click en el botón, modificamos el estado de counter
        setCounter(counter + 1);
      }}
    >
    Click me!
  </button>
</main>
);
}

export default App;

```

Fíjate como al cargar la página por primera vez, se ejecuta el `console.log`. Cada vez que modificas el **estado** `counter` haciendo *clic* en el botón, vuelve a ejecutarse el `console.log`.

Tip: Cada vez que se renderiza el componente, sale dos veces el `console.log` de "render". Esto es porque por defecto está activado el *React Strict Mode*, que, con el objetivo de detectar bugs en la aplicación, ejecuta los renderizados dos veces. Esta característica solo está habilitada en modo desarrollo.

```

render                                     App.jsx:7
render                                     installHook.js:1

```

Modificando estados que almacenan objetos

Imagina que el estado `counter` es un objeto, en vez de un simple número:

App.jsx

```

import { useState } from "react";

function App() {
  const [counter, setCounter] = useState({ value: 0, text: "galletas" });

  return (
    <main>
      <p>
        Hay {counter.value} {counter.text}
      </p>

      <button
        onClick={() => {
          // Sumarle uno al value de counter
        }}
      >
        Click me!
      </button>
    </main>
  );
}

```

```
    );  
  }  
  
  export default App;
```

Si quieres sumarle uno a la propiedad *value* del *counter*, podrías pensar en hacer algo así:

```
import { useState } from "react";  
  
function App() {  
  const [counter, setCounter] = useState({ value: 0, text: "galletas" });  
  
  return (  
    <main>  
      <p>  
        Hay {counter.value} {counter.text}  
      </p>  
  
      <button  
        onClick={() => {  
          // Sumarle uno al value de counter  
          counter.value++;  
  
          // Imprimimos por consola counter  
          console.log(counter);  
        }}  
      >  
        Click me!  
      </button>  
    </main>  
  );  
}  
  
export default App;
```

Si ejecutas este último código, verás como efectivamente por consola se muestra la propiedad *value* del *counter* actualizada correctamente, pero los cambios no se ven reflejados en la pantalla.

Igual que en el caso anterior, para modificar estados debes utilizar la función que retornada por *useState*, en este caso, *setCounter*. Si no utilizas dicha función, *React* no vuelve a renderizar el componente.

Puede parecer adecuado actualizar el estado de esta manera:

```
counter.value++;  
  
setCounter(counter);
```

Aquí, incrementas la propiedad `value` de `counter` y luego actualizas el **estado** con `setCounter`, pasando el objeto `counter` modificado.

Ahora bien, va a seguir sin funcionar. ¿Por qué? En el módulo de *JavaScript* vimos que un objeto siempre es el mismo, aunque sus contenidos cambien. Cuando llamas a `setCounter`, *React* es lo suficientemente listo como para detectar si realmente le estás enviando un valor nuevo para guardar en el **estado**, o si es el mismo que ya tenía. Si es el mismo, no realiza el cambio de estado, por lo que no vuelve a renderizar el componente.

A pesar de que has modificado el objeto `counter` y se lo has pasado a la función `setCounter`, *React* detecta que es exactamente el mismo objeto que tenía guardado antes, aunque sus contenidos hayan cambiado. Por este motivo, no se realiza el cambio de estado y no se vuelve a renderizar el componente.

Para arreglar este problema, debes pasarle a `setCounter` un objeto completamente nuevo.

App.jsx

```
import { useState } from "react";

function App() {
  const [counter, setCounter] = useState({ value: 0, text: "galletas" });

  return (
    <main>
      <p>
        Hay {counter.value} {counter.text}
      </p>

      <button
        onClick={() => {
          // Sumarle uno al value de counter
          counter.value++;

          // Cambiamos el estado por un objeto nuevo con los contenidos
          // actuales de counter
          setCounter({ ...counter });
        }}
      >
        Click me!
      </button>
    </main>
  );
}

export default App;
```

Al modificar estados que almacenan objetos, es necesario enviar siempre un objeto nuevo.

Tip: Con objetos, nos referimos a cualquier cosa que no sea un tipo primitivo, como por ejemplo arrays.

Ahora sí, verás como al cambiar el **estado** `counter`, el componente se vuelve a renderizar correctamente.

El **estado** del código anterior se puede actualizar de manera más limpia y concisa de la siguiente forma:

```
// Cambiamos el estado por un objeto nuevo con el value actualizado
setCounter({ ...counter, value: counter.value + 1 });
```

Consideraciones

- Un mismo componente puede tener tantos **estados** como necesites.
- Si llamas al mismo componente dos veces, cada instancia del componente tendrá sus propios **estados**. Por ejemplo, si creas un componente `Counter` con un estado que almacene el contador, al utilizar dicho componente varias veces, cada instancia tendrá su propio estado.
- Se crean **estados** para guardar información que necesitas que se actualice en pantalla al ser modificada.
- `useState` devuelve un array con dos valores: el *state actual* y una *función* mediante la cual actualizamos dicho *state* (`0, f`).



[Volver al índice principal](#)

Los formularios permiten que las aplicaciones web recojan información del usuario a través de campos de entrada. Crear formularios en React es parecido a cómo se hacía en *HTML*, pero con algunas diferencias clave, sobre todo a la hora de recoger y tratar los valores de los *inputs*.

El **state** en formularios de React

En JavaScript puro, para acceder al valor de un input, seleccionas el nodo DOM correspondiente y utilizas su propiedad **value**. Sin embargo, en React, los valores de los inputs generalmente se manejan a través de **estados**. En React, existen dos enfoques principales para trabajar con inputs: **inputs controlados** e **inputs no controlados**.

Inputs no controlados

Por defecto, los *inputs* son *no controlados*. ¿Qué quiere decir esto? Que no es el desarrollador quien se encarga de controlar su valor, es el **navegador**. Cada vez que se pone el cursor en un *input* y se pulsa una tecla, el navegador se encarga de actualizar el *DOM* e ir asignando el valor correspondiente al *input*.

Veamos un ejemplo para entender cómo manejar el valor de un *input no controlado* en un estado de React. Considera el siguiente formulario en **App.jsx**:

App.jsx

```
function App() {
  return (
    <form>
      <label htmlFor="name">Nombre</label>
      <input id="name" name="name" required />

      <button>Enviar</button>
    </form>
  );
}

export default App;
```

Tip: Utilizamos **htmlFor** en lugar de **for** en la etiqueta **<label>** porque **for** es una palabra reservada en JavaScript.

Ahora, crea un **estado** de React llamado **name**, que comience como una cadena de texto vacía:

App.jsx

```
import { useState } from "react";

function App() {
  // Creamos el estado "name" que empieza siendo una string vacía
  const [name, setName] = useState("");
```

```

    return (
      <form>
        <label htmlFor="name">Nombre</label>
        <input id="name" name="name" required />

        <button>Enviar</button>
      </form>
    );
  }

  export default App;

```

El objetivo es actualizar este **estado** cada vez que el valor del input cambie. Esto se logra utilizando el evento **onChange**, que se activa con cada cambio en el input.

Dentro del manejador de eventos (*event handler*), como siempre, recibirás por parámetro un objeto con información sobre el evento (llamémoslo **e**). Este objeto, tiene una propiedad **target**, tal como se ha visto en el módulo de *JavaScript*, es el elemento donde ha sucedido el evento. Es decir, si salta el evento **onChange**, porque alguien ha escrito un carácter en un *input*, el **target** va a ser dicho *input*.

Por último, los *inputs* tienen una propiedad **value** con el valor del *input*. Sabiendo esto, puedes acceder al **target** del objeto del evento (**e**) y acceder al **value** de dicho **target**. Esto te va a dar el valor que hay escrito en el *input* (**e.target.value**).

```

<input
  id="name"
  name="name"
  required
  onChange={(e) => {
    console.log(e.target.value);
  }}
/>

```

Si ejecutas este código, comprobarás como cada vez que realizas una modificación en el *input*, sale por consola el valor actualizado.

Ahora que ya sabes cómo acceder al valor del *input*, solo queda guardar dicho valor en el **estado** **name** creado anteriormente.

App.jsx

```

import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  // Imprimimos en consola el estado "name"

```

```

    console.log(name);

    return (
      <form>
        <label htmlFor="name">Nombre</label>
        <input
          id="name"
          name="name"
          required
          onChange={(e) => {
            // Cada vez que se produce un cambio en el input, cambiamos el estado
            "name" e introducimos el nuevo valor del input
            setName(e.target.value);
          }}
        />

        <button>Enviar</button>
      </form>
    );
  }

  export default App;

```

¡Listo! Ya tienes un estado que almacena el valor del *input* actualizado en todo momento.

Tip: Recuerda que cada vez que un estado cambia, se vuelve a renderizar el componente, por lo que el `console.log(name)` del ejemplo anterior, va a ejecutarse cada vez que el estado `name` cambie, es decir, cada vez que se realice una modificación en el *input*.

Inputs controlados

Controlar los *inputs* es la recomendación oficial por parte de los desarrolladores de *React*. Esto no quiere decir que puedas dejarlos sin controlar, pero deberías controlarlos siempre que puedas, ya que tiene sus ventajas.

Como se ha visto antes, por defecto los *inputs* son controlados por el navegador, no por el desarrollador. La idea de los *inputs controlados* es que el desarrollador tenga el control con *React*, y no el navegador.

Para controlar un *input*, tienes que darle su valor y encargarte de actualizarlo. Para darle un valor a *input*, puedes utilizar el atributo `value`. Observa el siguiente ejemplo:

App.jsx

```

import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  return (
    <form>

```

```

    <label htmlFor="name">Nombre</label>
    {/* Le aplicamos nosotros el value al input */}
    <input id="name" name="name" required value={name} />

    <button>Enviar</button>
  </form>
);
}

export default App;

```

En este ejemplo, podría considerarse que el *input* "name" es controlado, ya que le estás asignando su valor (`value={name}`). Ahora bien, si lo dejas así, ¿qué va a pasar? Que el valor del *input* va a ser siempre el **estado** `name`. Tal como está el código, ¿el estado `name` cambia alguna vez? No, nunca. Esto implica que el valor del *input* nunca va a cambiar, va a ser siempre una *string* vacía.

Si intentas escribir en el *input*, no será posible, ya que se le ha quitado el control al navegador y le estás diciendo al *input* que su valor es una *string* vacía, es decir, un texto vacío.

Para completar el ejemplo y hacerlo correctamente, falta actualizar el estado `name` cada vez que se produce el evento `onChange` en el *input*, justo lo que has hecho en el ejemplo del *input no controlado*.

App.jsx

```

import { useState } from "react";

function App() {
  const [name, setName] = useState("");

  // Imprimimos en consola el estado "name"
  console.log(name);

  return (
    <form>
      <label htmlFor="name">Nombre</label>
      <input
        id="name"
        name="name"
        required
        {/* Le aplicamos el value al input */}
        value={name}
        onChange={(e) => {
          // Cada vez que se produce un cambio en el input, cambiamos el estado
          "name" e introducimos el nuevo valor del input
          setName(e.target.value);
        }}
      />

      <button>Enviar</button>
    </form>
  );
}

```

```
    );  
  }  
  
  export default App;
```

Ahora sí que va a funcionar, ya que cuando el usuario introduce una letra en el input, salta el evento `onChange` y se actualiza el estado. Al actualizarse el estado, se vuelve a renderizar el componente con el nuevo `value` del `input`.

Importante: Los `inputs` de `type file` no se pueden controlar. Se suelen gestionar con un estado como hemos visto en la sección de `inputs no controlados` o con una `ref`, que lo verás un poco más abajo.

Ventajas de controlar los `inputs`

Cuando controlas los `inputs` como en el último ejemplo, tienes el control total y, además, sabes que el valor de tu estado y del `input` van a ser **siempre** iguales.

- Si tu estado cambia por cualquier motivo, se vuelve a renderizar el componente y el `input` muestra el nuevo `value`.
- Si se escribe en el `input`, se activa el evento `onChange`, que actualiza tu estado y vuelve a renderizar el componente.

En los `inputs no controlados` esto no sucede. Puede darse el caso de que el **estado** esté desincronizado con el valor del `input`. Veamos el siguiente ejemplo de un `input no controlado`:

App.jsx

```
import { useState } from "react";  
  
function App() {  
  const [name, setName] = useState("");  
  
  // Imprimimos en consola el estado "name"  
  console.log(name);  
  
  return (  
    <form>  
      <label htmlFor="name">Nombre</label>  
      <input  
        id="name"  
        name="name"  
        required  
        onChange={(e) => {  
          setName(e.target.value);  
        }}  
      />  
  
      <button
```

```

        type="button"
        onClick={() => {
            setName("pepe");
        }}
    >
        Cambiar nombre a Pepe
    </button>
    <button>Enviar</button>
</form>
);
}

export default App;

```

El **estado** empieza estando sincronizado con el valor del *input*, es decir, ambos son una *string* vacía. Si se escribe en el *input*, salta el evento `onChange` y el **estado** se actualiza. Siguen estando sincronizados.

Ahora bien, si haces *clic* en el botón de "Cambiar nombre a Pepe", el **estado** va a pasar a ser la *string* "Pepe". Sin embargo, el valor visible en el campo del *input* no cambiará. Esto sucede porque no estás asignando el valor del estado al *input* mediante la propiedad `value={name}`. En este caso, el navegador es quien gestiona el valor del *input*, no el estado de React.

Puedes comprobar todo esto si ejecutas el código en el navegador y te fijas en el valor del estado `name` que va saliendo por consola.

Almacenar datos de formulario en el `state` como un objeto

Si tienes un formulario con varios campos, puedes necesitar un **estado** para cada campo. Esto no está mal, pero para optimizar y mejorar la legibilidad del código, se suele crear **un solo estado**. Este **estado**, será un objeto que contiene los **valores de todos los *inputs*** del formulario.

SimpleForm.jsx

```

import { useState } from "react";

const SimpleForm = () => {
    // Inicializamos el estado del formulario con un objeto que contiene dos
    // campos: username y email.
    const [formState, setFormState] = useState({ username: "", email: "" });

    // Desestructuramos el target del event
    // Función para manejar los cambios en los campos de entrada del formulario
    const handleInputChange = ({ target }) => {
        // De target nos interesa dos valores: name (necesario declararlo en los
        // inputs del formulario) y value
        const { name, value } = target;

        // Actualizamos el estado del formulario agregando o modificando el campo
        // correspondiente.
        setFormState({ ...formState, [name]: value });
    };
}

```

```

};

// Imprimimos por consola el estado de los inputs
console.log(formState);

return (
  <>
    <h1>Simple form</h1>
    <input
      type="text"
      placeholder="Introduce tu nombre"
      // el valor del atributo name debe coincidir con una propiedad del
objeto
      name="username"
      // el valor actual para la propiedad correspondiente en el state
      value={formState.username}
      onChange={handleInputChange}
    />
    <input
      type="email"
      placeholder="Introduce tu email"
      name="email"
      value={formState.email}
      onChange={handleInputChange}
    />
  </>
);
};

export default SimpleForm;

```

Si observas la consola del navegador, verás como las diferentes propiedades del estado `formState` se van actualizando con los valores de los *inputs*.

Ten en cuenta que es necesario declarar el atributo `name` en los *inputs* para diferenciarlos, además, el valor de `name` no debe estar duplicado en el formulario.

Considera lo siguiente:

- **Estado unificado:** El componente `SimpleForm` utiliza un único objeto de estado (`formState`) para almacenar los valores de todos los campos del formulario.
- **Función de manejo general:** Utilizas una única función `handleInputChange` para manejar los cambios en cualquier campo de entrada del formulario.
- **Input controlado:** Cada campo de entrada es un *input controlado*, cuyo valor se establece y se actualiza en base al estado del componente.
- **Uso del atributo `name`:** Cada campo de entrada tiene un atributo `name`, cuyo valor debe ser único y coincidir con una de las propiedades del objeto de **estado**.